

Pre-lab Exercises

Exercise 1:

Measure the link lengths and fill out the following DH table using the QArm values:

Link	θ_i	d_i	a_i	α_i
1				
2				
3				
4				

Exercise 2: Implement the forward kinematics function `forward_kin(phi)` using your DH table. Test your function against the expected outputs below, debug any issues that arise, and record your results for the test cases:

Input (degrees)	Location [x, y, z]	Rotation Matrix
[0, 0, 0, 0]	[0.45, 0.0, 0.49]	$\begin{pmatrix} 0.0 & 0.0 & 1.0 \\ 0.0 & -1.0 & 0.0 \\ 1.0 & 0.0 & 0.0 \end{pmatrix}$
[30, 55, -60, 0]	[-0.08, 0.5, 0.86]	$\begin{pmatrix} -0.08 & 0.5 & 0.86 \\ -0.04 & -0.87 & 0.5 \\ 1.0 & 0.0 & 0.09 \end{pmatrix}$

Record your calculated outputs for the following configurations:

Input	Location [x, y, z]	Rotation Matrix
[0, -11, -75, 0]		
[-90, 73.5, -75, -50]		
[-90, 73.5, -75, 0]		
[90, -85, 75, 90]		

Exercise 3:

Answer the following conceptual questions on velocity kinematics:

1. Why can't we directly command a robotic manipulator with Cartesian velocities in task space $(\dot{x}, \dot{y}, \dot{z})$ without an inverse Jacobian ($\underline{\dot{q}} = \mathbf{J}^{-1}\underline{\dot{v}}$)? What physical variables do the robot's joint motors actually control?
2. For our 4-DOF QArm, why does restricting our task space to four variables (p_x, p_y, p_z, φ) produce a square 4×4 analytic Jacobian matrix, and what computational benefit does this provide for real-time trajectory tracking compared to an arbitrary 6-DOF geometric Jacobian?

In-Lab Exercises

Exercise 4:

Place the duck in zone 1. Manually move the duck into zone 2, release, then pick up and move to zone 3. Fill out joint positions at each point of interest as you go:

		Position			
		home	1	2	3
Joint	base				
	shoulder				
	elbow				
	wrist				

Exercise 5: A trajectory with defined knot points is provided in `trajectory_generator.py`. Implement trajectory generation using the provided Jacobian generator function and `scipy.interpolate.CubicSpline`. The knot points are formatted as 2D NumPy arrays, where each sub-array contains three values representing the (x,y,z) coordinates expressed in the base $\{0\}$ frame.

The final result should have the arm smoothly sweep across the top of the barrier wall. When completed, record the result (for submission) and show your working demonstration to the TA.

Post-lab Exercises

Exercise 6:

Explore what happens if we “artificially” introduce some error in accuracy or precision. This can manifest in several ways:

- Joint angle error due to incorrect measurement or backlash/slip
- Link measurement error due to manufacturing issues or material strain
- Signal or sensor noise

Calculate and record the task space positional error (magnitude) in several poses when we purposefully introduce error using your forward kinematics implementation.

Pose_1 = [0, 0, 0, 0] (home)

Pose_2 = [0, 55, -60, 0] (extended)

Pose_3 = [0, -80, 70, 0] (retracted)

Error Type	Pose 1 Positional Error	Pose 2 Positional Error	Pose 3 Positional Error
Shoulder joint + 10°			
Elbow joint + 10°			
L2 length + 5cm			
L4 length + 5cm			

Question: Identify which joint and link errors cause the largest overall positional discrepancy. Why does an error at one specific joint or link scale differently than another? Based on this behavior, what is one additional distinct pose you would test to explore the absolute worst-case positional error?

Exercise 7:

Answer the following synthesis questions:

1. While cubic spline trajectories are convenient and straightforward to implement, what are their main mechanical and kinematic drawbacks? (Consider higher-order derivatives such as jerk at knot points and the physical strain on robot actuators).
2. In the `integration_err.py` demo, open-loop numerical integration caused the arm’s trajectory to drift over time. Explain what causes this cumulative error in digital systems, and explain how the feedback term in `v_cmd = spline_velocity(t) + Kp * (spline(t) - location)` counteracts it.